BlueXP automation catalog

NetApp Automation

NetApp
June 08, 2023

This PDF was generated from https://docs.netapp.com/us-en/netapp-automation/solutions/bac-overview.html on June 08, 2023. Always check docs.netapp.com for the latest.

Table of Contents

BlueXP automation catalog . . . 1
Overview of the BlueXP automation catalog . . . 1
Azure Netapp Files . . . 1
Amazon FSx for NetApp . . . 7

BlueXP automation catalog

Overview of the BlueXP automation catalog

The BlueXP automation catalog is a collection of automation solutions available to NetApp customers, partners, and employees. Some of the features and benefits of the catalog are described below.

Single location for your automation needs

You can access the automation catalog through the BlueXP web user interface. This provides a single location for the scripts, playbooks, and modules needed to enhance the automation and operation of your NetApp products and services.

Scripts are created and tested by NetApp

All the automation solutions and scripts have been created and tested by NetApp. Each solution targets a specific customer use case or request. Most focus on integration with the NetApp file and data services.

Solution documentation

Each of the automation solutions includes associated documentation to help you get started. While the solutions are accessed through the BlueXP web interface, all the documentation is available at this site. The documentation is organized based on the NetApp products and cloud services.

Solid foundation for the future

NetApp is committed to helping our customers improve and streamline the automation of their data centers and cloud environments. We expect to continue enhancing the BlueXP automation catalog to address customer requirements, technology changes, and continued product integration.

We want to hear from you

The NetApp Customer Experience Office (CXO) automation team would like to hear from you. If you have any feedback, issues, or feature requests, please send an email to CXO automation team.

Azure Netapp Files

Install Oracle using Azure NetApp Files

You can use this automation solution to provision Azure NetApp File (ANF) volumes and install Oracle on an available virtual machine. Oracle then uses the volumes for data storage.

About this solution

At a high level, the automation code provided with this solution performs the following actions:

- Set up a NetApp account on Azure
- Set up a storage capacity pool on Azure
- Provision the Azure NetApp Files volumes based on the definition
- Create the mount points
- Mount the Azure NetApp Files volumes to the mount points

- Install Oracle on the Linux server
- Create the listeners and database
- Create the Pluggable Databases (PDBs)
- Start the listener and Oracle instance
- Install and configure the `azacsnap` utility to take a snapshot

Before you begin

You must have the following to complete the installation:

- The automation solution packaged as file `Ora_ANF_Prov.zip`. You can download the solution file at the BlueXP web user interface.
- A Linux VM with the following characteristics:
  - RHEL 8 (Standard_D8s_v3-RHEL-8)
  - Deployed on the same Azure Virtual Network used for the Azure NetApp Files provisioning
- Azure account

The automation solution is provided as an image and run using Docker and Docker Compose. You need to install both of these on the Linux virtual machine as described below.

You should also register the VM with RedHat using the command `sudo subscription-manager register`. The command will prompt you for your account credentials. If needed, you can create an account at https://developers.redhat.com/.

Step 1: Install and configure Docker

Install and configure Docker in a RHEL 8 Linux virtual machine.

Steps

1. Install the Docker software using the following commands.

   ```
   dnf config-manager --add
   -repo=https://download.docker.com/linux/centos/docker-ce.repo
   dnf install docker-ce --nobest -y
   ```

2. Start Docker and display the version to confirm the installation was successful.

   ```
   systemctl start docker
   systemctl enable docker
   docker --version
   ```

3. Add the required Linux group with an associated user.

   First check if the group **docker** exists in your Linux system. If it doesn't, create the group and add the user. By default, the current shell user is added to the group.

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

4. Activate the new group and user definitions

   If you created a new group with a user, you need to activate the definitions. To do this, you can logout of Linux and then back in. Or you can run the following command.

```
newgrp docker
```

Step 2: Install Docker Compose and the NFS utilities

Install and configure Docker Compose along with the NFS utilities package.

Steps

1. Install Docker Compose and display the version to confirm the installation was successful.

```
dnf install curl -y
curl -L
"https://github.com/docker/compose/releases/download/1.29.2/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
docker-compose --version
```

2. Install the NFS utilities package.

```
sudo yum install nfs-utils
```

Step 3: Download the Oracle installation files

Download the required Oracle installation and patch files.

Steps

1. Sign in to your Oracle account as needed.
2. Download the following files.

File	Description
LINUX.X64_193000_db_home.zip	19.3 base installer
p31281355_190000_Linux-x86-64.zip	19.8 RU patch
p6880880_190000_Linux-x86-64.zip	opatch version 12.2.0.1.23
azacsnap_installer_v5.0.run	azacsnap installer

3. Place all the installation files in the folder `/tmp/archive`.
4. Make sure all users on the database server have full access (read, write, execute) to the folder `/tmp/archive`.

Step 4: Prepare the Docker image

You need to extract and load the Docker image provided with the automation solution.

Steps

1. Copy the solution file `Ora_ANF_Prov.zip` to the virtual machine where the automation code will run.

   ```
   scp -i ~/<private-key.pem> -r Ora_ANF_Prov.zip user@<IP_ADDRESS_OF_VM>
   ```

   The input parameter `private-key.pem` is your private key file used for Azure virtual machine authentication (Oracle/Ansible host).

2. Navigate to the correct folder with the solution file and unzip the file.

   ```
   unzip Ora_ANF_Prov.zip
   ```

3. Navigate to the new folder `Ora_ANF_Prov` created with the unzip operation and list the files. You should see file `Ora_ANF_Prov_image_latest.tar.gz`.

   ```
   ls -lt
   ```

4. Load the Docker image file. The load operation should normally complete in a few seconds.

   ```
   docker load -i Ora_ANF_Prov_image_latest.tar.gz
   ```

5. Confirm the Docker image is loaded.

   ```
   docker images
   ```

   You should see the Docker image `ora_anf_prov_image` with the tag `latest`.

   ```
      REPOSITORY          TAG       IMAGE ID      CREATED      SIZE
   ora_anf_prov_image    latest   ay98y7853769   1 week ago   2.58GB
   ```

Step 5: Create an external volume

You need an external volume to make sure the Terraform state files and other important files are persistent. These files must be available for Terraform to run the workflow and deployments.

Steps

1. Create an external volume outside of Docker Compose.

   Make sure to update the volume name before running the command.

   ```
   docker volume create <VOLUME_NAME>
   ```

2. Add the path to the external volume to the `.env` environment file using the command:

   `PERSISTENT_VOL=path/to/external/volume:/ora_anf_prov`.

   Remember to keep the existing file contents and colon formatting. For example:

   ```
   PERSISTENT_VOL= ora_anf _volume:/ora_anf_prov
   ```

3. Update the Terraform variables.
   a. Navigate to the folder `ora_anf_variables`.
   b. Confirm the following two files exist: `terraform.tfvars` and `variables.tf`.
   c. Update the values in `terraform.tfvars` as required for your environment.

Step 6: Install Oracle

You can now provision and install Oracle.

Steps

1. Install Oracle using the following sequence of commands.

   ```
   docker-compose up terraform_ora_anf
   bash /ora_anf_variables/setup.sh
   docker-compose up linux_config
   bash /ora_anf_variables/permissions.sh
   docker-compose up oracle_install
   ```

2. Reload your Bash variables and confirm by displaying the value for `ORACLE_HOME`.
   a. `cd /home/oracle`
   b. `source .bash_profile`
   c. `echo $ORACLE_HOME`
3. You should be able to login to Oracle.

   ```
   sudo su oracle
   ```

Step 7: Validate the Oracle installation

You should confirm the Oracle installation was successful.

Steps

1. Log in to the Linux Oracle server and display a list of the Oracle processes. This confirms the installation completed as expected and the Oracle database is running.

```
ps -ef | grep ora
```

2. Log in to the database to examine the database configuration and to confirm the PDBs were created properly.

```
sqlplus / as sysdba
```

You should see output similar to the following:

```
SQL*Plus: Release 19.0.0.0.0 - Production on Thu May 6 12:52:51 2021
Version 19.8.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.8.0.0.0
```

3. Execute a few simple SQL commands to confirm the database is available.

```
select name, log_mode from v$database;
show pdbs.
```

Step 8: Install the azacsnap utility and perform a snapshot backup

You need to install and run the `azacsnap` utility to perform a snapshot backup.

Steps

1. Install the container.

```
docker-compose up azacsnap_install
```

2. Switch to the snapshot user account.

```
su - azacsnap
execute /tmp/archive/ora_wallet.sh
```

3. Configure a storage backup detail file. This will create the `azacsnap.json` configuration file.

```
cd /home/azacsnap/bin/
azacsnap -c configure --configuration new
```

4. Perform a snapshot backup.

```
azacsnap -c backup --other data --prefix ora_test --retention=1
```

Step 9: Optionally migrate an on-premise PDB to the cloud

You can optionally migrate the on-premise PDB to the cloud.

Steps

1. Set the variables in the `tfvars` files as needed for your environment.
2. Migrate the PDB.

```
docker-compose -f docker-compose-relocate.yml up
```

Amazon FSx for NetApp

Amazon FSx for NetApp - Burst to cloud

You can use this automation solution to provision Amazon File Services for NetApp with volumes and an associated FlexCache.

> For convenience and brevity, Amazon File Services for NetApp will be referred to as **FSxN**.

About this solution

At a high level, the automation code provided with this solution performs the following actions:

- Provision a destination FSxN file system
- Provision Storage Virtual Machines (SVMs) for the file system
- Create a cluster peering relationship between the source and destination systems
- Create an SVM peering between the source system and destination system for FlexCache
- Optionally create FlexVol volumes in FSxN
- Create a FlexCache volume in FSxN with the source pointing to on-prem storage

The automation is based on Docker and Docker Compose which must be installed on the Linux virtual machine as described below.

Before you begin

You must have the following to complete the provisioning and configuration:

- The automation solution packaged as file `AWS_FSxN_BTC.zip`. You can download the file at the BlueXP web user interface.
- Network connectivity between the source and destination systems.
- A Linux VM with the following characteristics:
  - Debian-based Linux distribution
  - Deployed on the same VPC subset used for FSxN provisioning
- AWS account.

Step 1: Install and configure Docker

Install and configure Docker in a Debian-based Linux virtual machine.

Steps

1. Prepare the environment.

```
sudo apt-get update
sudo apt-get install apt-transport-https ca-certificates curl gnupg-
agent software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key
add -
sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
sudo apt-get update
```

2. Install Docker and verify the installation.

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
docker --version
```

3. Add the required Linux group with an associated user.

   First check if the group **docker** exists in your Linux system. If it doesn't, create the group and add the user. By default, the current shell user is added to the group.

```
sudo groupadd docker
sudo usermod -aG docker $(whoami)
```

4. Activate the new group and user definitions

If you created a new group with a user, you need to activate the definitions. To do this, you can logout of Linux and then back in. Or you can run the following command.

```
newgrp docker
```

Step 2: Install Docker Compose

Install Docker Compose in a Debian-based Linux virtual machine.

Steps

1. Install Docker Compose.

```
sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

2. Verify the installation was successful.

```
docker-compose --version
```

Step 3: Prepare the Docker image

You need to extract and load the Docker image provided with the automation solution.

Steps

1. Copy the solution file `AWS_FSxN_BTC.zip` to the virtual machine where the automation code will run.

```
scp -i ~/<private-key.pem> -r AWS_FSxN_BTC.zip user@<IP_ADDRESS_OF_VM>
```

   The input parameter `private-key.pem` is your private key file used for AWS virtual machine authentication (EC2 instance).

2. Navigate to the correct folder with the solution file and unzip the file.

```
unzip AWS_FSxN_BTC.zip
```

3. Navigate to the new folder `AWS_FSxN_BTC` created with the unzip operation and list the files. You should see file `aws_fsxn_flexcache_image_latest.tar.gz`.

```
ls -la
```

4. Load the Docker image file. The load operation should normally complete in a few seconds.

```
docker load -i aws_fsxn_flexcache_image_latest.tar.gz
```

5. Confirm the Docker image is loaded.

```
docker images
```

You should see the Docker image `aws_fsxn_flexcache_image` with the tag `latest`.

```
   REPOSITORY              TAG     IMAGE ID      CREATED      SIZE
aws_fsxn_flexcahce_image  latest  ay98y7853769  2 weeks ago  1.19GB
```

Step 4: Create environment file for AWS credentials

You must create a local variable file for authentication using the access and secret key. Then add the file to the `.env` file.

Steps

1. Create the `awsauth.env` file in the following location:

   `path/to/env-file/awsauth.env`

2. Add the following content to the file:

```
access_key=<>
secret_key=<>
```

   The format **must** be exactly as shown above without any spaces between `key` and `value`.

3. Add the absolute file path to the `.env` file using the `AWS_CREDS` variable. For example:

   `AWS_CREDS=path/to/env-file/awsauth.env`

4. Navigate to the `aws_fsxn_variables` folder and update the access and secret key in the credentials file.

Step 5: Create an external volume

You need an external volume to make sure the Terraform state files and other important files are persistent. These files must be available for Terraform to run the workflow and deployments.

Steps

1. Create an external volume outside of Docker Compose.

   Make sure to update the volume name (last parameter) to the appropriate value before running the

command.

```
docker volume create aws_fsxn_volume
```

2. Add the path to the external volume to the `.env` environment file using the command:

   `PERSISTENT_VOL=path/to/external/volume:/volume_name`

   Remember to keep the existing file contents and colon formatting. For example:

   ```
   PERSISTENT_VOL=aws_fsxn_volume:/aws_fsxn_flexcache
   ```

   You can instead add an NFS share as the external volume using a command such as:

   `PERSISTENT_VOL=nfs/mnt/document:/aws_fsx_flexcache`

3. Update the Terraform variables.
   a. Navigate to the folder `aws_fsxn_variables`.
   b. Confirm the following two files exist: `terraform.tfvars` and `variables.tf`.
   c. Update the values in `terraform.tfvars` as required for your environment.

      See Terraform resource: aws_fsx_ontap_file_system for more information.

Step 6: Provision Amazon FSx for NetApp and FlexCache

You can now provision Amazon FSx for NetApp and FlexCache.

Steps

1. Navigate to the folder root (AWS_FSXN_BTC) and issue the provisioning command.

   ```
   docker-compose -f docker-compose-provision.yml up
   ```

   This command creates two containers. The first container deploys FSxN and the second container creates the cluster peering, SVM peering, destination volume, and FlexCache.

2. Monitor the provisioning process.

   ```
   docker-compose -f docker-compose-provision.yml logs -f
   ```

   This command gives you the output in real-time, but has been configured to capture the logs through the file `deployment.log`. You can change the name of these log files by editing the `.env` file and updating the variables `DEPLOYMENT_LOGS`.

Step 7: Destroy Amazon FSx for NetApp and FlexCache

You can optionally delete and remove Amazon FSx for NetApp and FlexCache.

1. Set the variable `flexcache_operation` in the `terraform.tfvars` file to "destroy".
2. Navigate to the folder root (AWS_FSXN_BTC) and issue the following command.

   ```
   docker-compose -f docker-compose-destroy.yml up
   ```

   This command creates two containers. The first container delete FlexCache and the second container delete's Amazon FSx for NetApp.

3. Monitor the provisioning process.

   ```
   docker-compose -f docker-compose-destroy.yml logs -f
   ```

Copyright information

Copyright © 2023 NetApp, Inc. All Rights Reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means—graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system—without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

LIMITED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (b)(3) of the Rights in Technical Data -Noncommercial Items at DFARS 252.227-7013 (FEB 2014) and FAR 52.227-19 (DEC 2007).

Data contained herein pertains to a commercial product and/or commercial service (as defined in FAR 2.101) and is proprietary to NetApp, Inc. All NetApp technical data and computer software provided under this Agreement is commercial in nature and developed solely at private expense. The U.S. Government has a non-exclusive, non-transferrable, nonsublicensable, worldwide, limited irrevocable license to use the Data only in connection with and in support of the U.S. Government contract under which the Data was delivered. Except as provided herein, the Data may not be used, disclosed, reproduced, modified, performed, or displayed without the prior written approval of NetApp, Inc. United States Government license rights for the Department of Defense are limited to those rights identified in DFARS clause 252.227-7015(b) (FEB 2014).

Trademark information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.